Verificación de presupuestos

Asadito RG · Implementación y guía de Java · 13 de noviembre de 2026

1. Qué se incorporó

El ZIP compartido se incorporó como un primer commit, conservando su código. La integración posterior agrega el formulario de verificación, una página de resultados del usuario y la consulta de la verificación dentro del detalle de cada presupuesto. Se usa la sesión existente de Spring Security.

Ingresá a tu cuenta y abrí Mis presupuestos. En cada tarjeta o en el detalle aparece Verificar compra. Ingresá el total que realmente pagaste y, si querés, una observación. La comparación debe corresponder a los mismos cortes, cantidades, comercio y moneda; no incluyas compras adicionales ni envío.

Después de guardar, el resultado aparece en el detalle y en Mis resultados, accesible desde el menú. Esa página permite filtrar por comercio/número y por resultado, ver compras pendientes y comparar visualmente estimado y pagado. El resumen corresponde a los presupuestos cargados en esa página; los errores de consulta se distinguen de una verificación pendiente.

La verificación es una declaración del usuario sobre su compra. No representa una validación de un ticket por parte del comercio. Se permite una verificación por presupuesto; no se incorporan edición ni eliminación de verificaciones.

2. Archivos Java del ZIP

Archivo	Función
modelos/Verificaciones.java	Entidad con presupuesto, fecha, total real, diferencia, porcentaje, observación y valoración. Mantiene el vínculo único presupuesto_id.
repositorios/VerificacionesRepository.java	Búsquedas findByPresupuesto_Id y existsByPresupuesto_Id.
enums/ValoracionVerificacion.java	Valores ACORDE, SOBREESTIMADO y SUBESTIMADO.
controllers/VerificacionesController.java	GET y POST de la verificación de un presupuesto.
dtos/VerificacionesDTO.java	Respuesta JSON para el frontend.

Las rutas Java parten de src/main/java/com/appasadito/AsaditoRg/. Los modelos, el repositorio y el enum se mantienen como en el ZIP. También se incorporaron las dos líneas vacías que traía PreciosController; no cambian su comportamiento.

3. Ajustes necesarios al controlador

En VerificacionesController se conserva el enfoque de @Autowired, repositorios y un controlador directo. No se agrega otra capa de servicios para la verificación.

El POST del ZIP declaraba totalReal y observaciones con @PathVariable, pero esos datos no aparecen en la URL. Se reciben como formulario con @RequestParam:

```
@PostMapping("/presupuestos/{id}/verificacion")
public VerificacionesDTO agregarVerificacion(
    @PathVariable int id,
    @RequestParam double totalReal,
    @RequestParam(defaultValue = "") String observaciones,
    Authentication authentication)
```

El método conserva @Transactional y devuelve 201 al crear. El formulario envía totalReal y observaciones como application/x-www-form-urlencoded y lleva la cabecera CSRF de la sesión actual.

Se agrega buscarPresupuesto, un método privado que usa la búsqueda por ID y dueño que ya tenía tu repositorio:

```
presupuestosRepository.findByIdAndUsuario_Id(id, usuario.getId())
```

Se usa en GET y POST antes de consultar o crear una verificación. Conocer el número del presupuesto no permite ver ni escribir datos ajenos. Incluso una cuenta ADMIN recibe 404 para un presupuesto de otra persona. La autenticación por sí sola no reemplaza este control de propiedad.

El objeto nuevo se vincula con verificacion.setPresupuesto(presupuesto). Esa asignación faltaba en el ZIP: sin ella el registro no queda relacionado con la compra y no se recupera por presupuesto_id.

Se validan total pagado positivo y finito, hasta dos decimales, máximo 999999999999.99, observación de hasta 300 caracteres y un total estimado válido para dividir. Un presupuesto ya verificado devuelve 409. La restricción UNIQUE existente protege también ante dos solicitudes simultáneas.

El guardado usa saveAndFlush y luego registra VERIFICACION_CREADA en MovimientosService, dentro de la misma transacción. Si falla el guardado, no queda un movimiento de una verificación inexistente. El presupuesto original conserva sus importes.

4. Cálculo y valoración

Se usa el mismo criterio de tu enum y controlador: diferencia = pagado - estimado; porcentaje = diferencia / estimado × 100. Un resultado dentro del ±5 %, incluidos ambos límites, es ACORDE.

Estimado	Pagado	Resultado
$20.000	$20.800	ACORDE: +4 %
$20.000	$21.000	ACORDE: +5 %, límite incluido
$20.000	$18.000	SOBREESTIMADO: -10 %, se presupuestó de más
$20.000	$22.000	SUBESTIMADO: +10 %, se presupuestó de menos
$20.000	$21.000,01	SUBESTIMADO: supera el límite por un centavo

Para esta comparación se convierten los importes a BigDecimal y se redondea el estimado histórico a dos decimales. Esto coincide con los centavos que se muestran al usuario. El precio y los DTO originales del catálogo no se modifican; tampoco se reescribe el total del presupuesto guardado.

La clasificación compara los importes antes de redondear el porcentaje mostrado:

```
boolean acorde = diferencia.abs().multiply(new BigDecimal("100"))
    .compareTo(estimado.multiply(new BigDecimal("5"))) <= 0;
```

Se guarda el porcentaje con seis decimales y se muestran dos en la interfaz. Así, una diferencia apenas superior al 5 % puede mostrarse redondeada como 5,00 %, pero seguirá clasificada fuera del margen. La

página aclara que el porcentaje mostrado está redondeado.

5. Ajustes del DTO y los errores

VerificacionesDTO ahora devuelve presupuestoId y totalEstimado como valores simples. Se evita devolver la entidad Presupuestos completa: sus relaciones pueden producir ciclos JSON y problemas al acceder a relaciones LAZY fuera de la transacción.

Se conserva fecha, diferencia, porcentaje, observacion y valoracion. Se agrega la asignación totalReal = verificaciones.getTotalReal(), que faltaba en el constructor original; sin esa asignación se respondía cero aunque hubiera un pago guardado.

Ejemplo de la respuesta de GET o POST:

```
{
  "presupuestoId": 12,
  "totalEstimado": 20000.0,
  "fecha": "2026-11-13T18:30:00",
  "totalReal": 22000.0,
  "diferencia": 2000.0,
  "porcentaje": 10.0,
  "observacion": "Mismos cortes y kilos",
  "valoracion": "SUBESTIMADO"
}
```

En ApiErrors se cambia únicamente el texto general de HTTP 409 para que también sirva a las nuevas tablas: “No se pudo guardar por un conflicto con datos existentes. Actualizá la página antes de volver a intentarlo”. Se siguen ocultando detalles SQL.

No se cambian LocalesDTO, ProductosDTO, PreciosDTO ni ProductoPrecioDTO. No se agrega una dependencia Java ni se cambia el esquema de sesiones.

6. Contrato HTTP

Operación	Respuesta
GET /api/presupuestos/{id}/verificacion	200 con la verificación del dueño. 404 si aún no existe o si el presupuesto no pertenece a la cuenta.
POST en la misma ruta	201 con el resultado. Recibe totalReal y observaciones como formulario, con CSRF.
Datos inválidos	400. No se guarda verificación ni movimiento.
Presupuesto ajeno o inexistente	404. Se comprueba primero el dueño.
Segunda verificación	409. Se conserva la primera.
Sin sesión / sin CSRF	401 / 403, según la comprobación que corresponda.

El frontend crea resultados consultando el listado propio de presupuestos y la verificación de cada uno. Limita esas consultas a cuatro simultáneas. No se agrega un endpoint Java de resultados globales ni se expone el historial de otras cuentas.

7. Archivos del frontend

Archivo	Responsabilidad
resultados-presupuestos.html y resultados-presupuestos.js	Página del usuario: resumen, tarjetas comparativas, filtros, pendientes y acceso al detalle.
verificaciones-ui.js	Formulario compartido, consultas, confirmaciones flotantes y presentación del resultado.
verificaciones-modelo.js	Validación del JSON y resumen de los resultados disponibles.
verificaciones.css	Estilos del formulario, las tarjetas, las barras comparativas y el detalle en móvil.
mis-presupuestos.html y mis-presupuestos.js	Acción Verificar compra y datos de la verificación dentro del detalle. Acepta ?presupuesto=12 para abrir un presupuesto desde los resultados.
session.js y navegación	Muestra Mis resultados al ingresar y limpia los datos al cerrar sesión o cambiar de cuenta.

Las observaciones se muestran como texto escapado. Un fallo de consulta se muestra como resultado no disponible; no se informa falsamente que esté pendiente. Guardar una verificación no crea otro presupuesto. El PDF histórico sigue mostrando el presupuesto original; los datos de la verificación están en el detalle y en Mis resultados.

8. Pruebas y explicación para la materia

Se agrega VerificacionesTests en src/test, con casos de guardado/lectura, las tres valoraciones, límites exactos del 5 %, un centavo fuera del margen, propiedad del presupuesto, duplicados, importes inválidos, longitud de observaciones y CSRF. Son pruebas de la aplicación, no clases que se despliegan como controladores.

Pasaron 19 pruebas Java en total, las pruebas de lógica del frontend y la comprobación del navegador contra el backend real con una base H2 temporal. En esta última se guardaron presupuestos, se verificaron tres compras, se filtraron resultados, se consultaron observaciones desde el detalle y se comprobó que otra cuenta recibiera 404. También se probó la regeneración del PDF sin cambiar ni duplicar el presupuesto original.

Para explicarlo: seguí POST de formulario → usuario de la sesión → búsqueda del presupuesto propio → validaciones → cálculo → entidad vinculada → repositorio → movimiento → DTO → frontend. La autorización del dueño y el cálculo definitivo están en Java; la presentación, filtros y gráficos pequeños quedan en JavaScript.

La comparación depende de que el usuario informe una compra equivalente. Comprobantes adjuntos, edición de verificaciones, confirmación por el comercio y analítica entre usuarios quedan para otra etapa.